

DOCUMENTAÇÃO DE PROJETO

Gerenciamento de Armazenamento AWS

Automação de Snapshots EBS com cron e Python + Sincronização S3 com Política de Retenção e Recuperação por Versionamento

Parâmetro	Detalhe
Serviços principais	Amazon EBS + Amazon S3 + Amazon EC2
Ambiente	Duas instâncias: Command Host (administração) e Processor (volume gerenciado)
Escopo técnico	Automação de backups, política de retenção com Python, sync S3 e recuperação por versionamento
Duração aproximada	45 minutos

1. Problema / Contexto

Criar snapshots manualmente é suficiente para um ambiente de laboratório — mas inviável para produção. Em ambientes reais, volumes EBS precisam de snapshots regulares e automáticos, com uma política de retenção que descarte backups antigos para controlar custos. Sem automação, a disciplina de backup depende de intervenção humana constante; sem retenção, o custo dos snapshots acumulados cresce indefinidamente.

No lado do armazenamento de objetos, o Amazon S3 é frequentemente usado como destino de sincronização de arquivos gerados por instâncias EC2 — logs, artefatos de build, dados processados. O comando `aws s3 sync` com a flag `--delete` é poderoso para manter o bucket em sincronia com o diretório local, mas carrega um risco específico: ele propaga exclusões. Se um arquivo for removido localmente por engano e o sync for executado em seguida, o arquivo desaparece do bucket também.

Este projeto endereçou as duas frentes de forma prática: a automação completa de snapshots EBS com política de retenção implementada em Python, e o ciclo de sincronização S3 com demonstração real de perda e recuperação de arquivo via versionamento — o mecanismo que torna o S3 resiliente a exclusões acidentais.

2. Objetivo

Meu objetivo técnico foi implementar duas camadas de proteção de dados operando em conjunto: automação de backup de volumes EBS com retenção controlada, e sincronização de arquivos com S3 protegida por versionamento. De forma específica:

- Criar snapshots do volume EBS da instância Processor de forma manual e automatizada via cron, executando todos os comandos CLI a partir da Command Host.

- Executar o script Python `snapshotter_v2.py` para aplicar uma política de retenção que mantém apenas os dois snapshots mais recentes, excluindo automaticamente os demais.
- Sincronizar arquivos da instância Processor com um bucket S3 habilitado com versionamento, utilizando `aws s3 sync` com e sem a flag `--delete`.
- Demonstrar e executar o fluxo completo de recuperação de um arquivo excluído acidentalmente do S3, utilizando `list-object-versions` e `get-object` com `VersionId` específico.

3. Solução

A solução foi estruturada em duas frentes independentes mas complementares, operando sobre o mesmo ambiente de duas instâncias. A arquitetura do ambiente antes de qualquer operação:

Instância	Função	Acesso configurado
Command Host	Administração remota — executa todos os comandos CLI de snapshot sobre a Processor	AWS CLI pré-configurada para gerenciar recursos da conta
Processor	Instância operacional — possui o volume EBS gerenciado e executa o <code>s3 sync</code>	Perfil IAM <code>S3BucketAccess</code> associado para acesso ao S3 e EBS

3.1 Snapshots EBS — Manual, Automático e Política de Retenção

Toda a administração de snapshots foi executada a partir da Command Host via AWS CLI. O fluxo correto para um snapshot consistente exige parar a instância antes da captura — garantindo que nenhuma escrita esteja em curso no volume no momento do snapshot.

```
# Passo 1 — Identificar o VolumeId do EBS da instância Processor pela tag Name
aws ec2 describe-instances \
  --filter 'Name=tag:Name,Values=Processor' \
  --query 'Reservations[0].Instances[0].BlockDeviceMappings[0].Ebs.
{VolumeId:VolumeId}'

# Passo 2 — Obter o InstanceId da Processor
aws ec2 describe-instances \
  --filters 'Name=tag:Name,Values=Processor' \
  --query 'Reservations[0].Instances[0].InstanceId'

# Passo 3 — Parar a instância e aguardar estado stopped (waiter bloqueia o
terminal)
aws ec2 stop-instances --instance-ids <INSTANCE-ID>
aws ec2 wait instance-stopped --instance-id <INSTANCE-ID>

# Passo 4 — Criar o snapshot e aguardar conclusão antes de reiniciar
aws ec2 create-snapshot --volume-id <VOLUME-ID>
aws ec2 wait snapshot-completed --snapshot-id <SNAPSHOT-ID>

# Passo 5 — Reiniciar a instância
aws ec2 start-instances --instance-ids <INSTANCE-ID>
```

Por que parar a instância antes do snapshot: Com a instância rodando, pode haver operações de escrita em andamento no volume no momento da captura — resultando em um snapshot com dados em estado intermediário (inconsistente). Parar a instância garante que o volume está quiesced: nenhuma escrita pendente, snapshot íntegro. Em workloads onde parar a instância não é viável, o Amazon Data Lifecycle Manager com agentes de consistência de aplicação é a alternativa recomendada.

aws ec2 wait — o padrão-ouro para scripts de infraestrutura assíncronos: Os comandos stop-instances, create-snapshot e start-instances são assíncronos: retornam imediatamente sem esperar a conclusão da operação. O waiter bloqueia o terminal até que o estado desejado seja atingido — sem loops manuais de polling. Qualquer script de infraestrutura que encadeia operações dependentes de estado deve usar waiters para garantir a sequência correta.

3.2 Automação com cron — Snapshots a Cada Minuto

Para gerar rapidamente múltiplos snapshots e testar a política de retenção, configurei um cron job na Command Host que executa o create-snapshot a cada minuto. A sintaxe do cron é composta por cinco campos posicionais seguidos do comando:

Campo 1	Campo 2	Campo 3	Campo 4	Campo 5	Comando
Minuto (0-59)	Hora (0-23)	Dia do mês (1-31)	Mês (1-12)	Dia da semana (0-7)	Ação a executar
*	*	*	*	*	A cada minuto (lab)
0	0	*	*	*	Diariamente à meia-noite (produção)

```
# Criar o arquivo de definição do cron job
echo "* * * * * aws ec2 create-snapshot --volume-id <VOLUME-ID> 2>&1 >>
/tmp/cronlog" > cronjob

# Registrar o cron job no crontab do usuário
crontab cronjob

# Aguardar alguns minutos e verificar os snapshots acumulados
aws ec2 describe-snapshots --filters "Name=volume-id,Values=<VOLUME-ID>"

# Remover o cron job após acumular snapshots suficientes para o teste
crontab -r
```

cron vs. AWS Data Lifecycle Manager em produção: O cron é funcional para automações simples em um único host, mas tem limitações sérias para gestão de snapshots em escala: não tem visibilidade centralizada, não gerencia retenção nativamente e falha silenciosamente se a instância estiver parada. O AWS Data Lifecycle Manager é a solução gerenciada que resolve todos esses pontos — políticas de retenção configuráveis, execução independente de estado da instância e integração nativa com CloudWatch para monitoramento.

3.3 Política de Retenção com snapshotter_v2.py

Com vários snapshots acumulados pelo cron, executei o script Python `snapshotter_v2.py` na Command Host. O script implementa uma política de retenção completa: lista todos os volumes EBS da conta, cria um novo snapshot de cada um, ordena os snapshots existentes por data de criação e exclui todos exceto os dois mais recentes.

```
# Inspecionar o script antes de executar — boa prática com scripts de exclusão
more /home/ec2-user/snapshotter_v2.py

# Verificar quantos snapshots existem antes da execução
aws ec2 describe-snapshots \
  --filters "Name=volume-id,Values=<VOLUME-ID>" \
  --query 'Snapshots[*].SnapshotId'

# Executar a política de retenção
python3 snapshotter_v2.py

# Verificar o resultado: apenas 2 IDs devem restar
aws ec2 describe-snapshots \
  --filters "Name=volume-id,Values=<VOLUME-ID>" \
  --query 'Snapshots[*].SnapshotId'
```

Política de retenção aplicada com sucesso: De vários snapshots acumulados pelo cron, o script manteve exatamente os 2 mais recentes e excluiu todos os demais. O número de IDs retornados pelo `describe-snapshots` passou de múltiplos para 2 — confirmando a execução correta da política.

O padrão do `snapshotter_v2.py` é reutilizável: A lógica central do script — listar recursos, ordenar por timestamp, manter N mais recentes, excluir os demais — é um padrão aplicável a qualquer política de retenção em nuvem. O mesmo padrão serve para gerenciar versões de objetos no S3, imagens de container no ECR ou backups de banco de dados no RDS. Ajustar apenas o número de itens a manter e o serviço alvo.

3.4 Sincronização S3 — Upload, Exclusão e Recuperação por Versionamento

Esta foi a etapa de maior profundidade operacional do projeto, executada a partir da instância Processor com o perfil IAM `S3BucketAccess` associado. O fluxo percorreu quatro estados distintos do bucket: vazio → com 3 arquivos → com 2 arquivos (após exclusão com `--delete`) → recuperado com 3 arquivos via versionamento.

Antes de qualquer upload, habilitei o versionamento no bucket. Essa ordem é crítica: objetos enviados antes do versionamento ser ativado recebem `VersionId` null e não têm versões anteriores recuperáveis — o versionamento precisa estar ativo no momento do primeiro upload.

```
# Habilitar versionamento no bucket ANTES do primeiro upload
aws s3api put-bucket-versioning \
  --bucket <s3-bucket-name> \
```

```
--versioning-configuration Status=Enabled

# Upload inicial: sincronizar os 3 arquivos para o bucket
aws s3 sync files s3://<s3-bucket-name>/files/

# Confirmar os 3 arquivos no bucket
aws s3 ls s3://<s3-bucket-name>/files/

# Simular exclusão acidental do arquivo localmente
rm files/file1.txt

# Sync com --delete: propaga a exclusão local para o bucket
# file1.txt é removido do bucket e um delete marker é criado
aws s3 sync files s3://<s3-bucket-name>/files/ --delete

# Confirmar: apenas file2.txt e file3.txt permanecem visíveis
aws s3 ls s3://<s3-bucket-name>/files/
```

aws s3 sync sem --delete

Idempotente e seguro: envia apenas arquivos novos ou modificados. Nunca remove nada do destino. Ideal para backups acumulativos onde o historial precisa ser preservado.

aws s3 sync com --delete

Espelha o diretório local exatamente: remove do destino tudo que não existe mais na origem. Essencial para deploys de sites estáticos — perigoso em backups se uma exclusão local acidental for propagada.

Com o versionamento ativo, o S3 não destruiu o conteúdo de file1.txt ao receber a instrução de exclusão — criou um delete marker: um marcador especial que torna o objeto invisível nas listagens normais, mas preserva todas as versões anteriores intactas. A recuperação foi feita em três passos encadeados:

Passo	Comando	Objetivo
1	<code>aws s3api list-object-versions</code>	Listar todas as versões e delete markers de file1.txt para obter o VersionId da versão original
2	<code>aws s3api get-object --version-id <ID></code>	Baixar especificamente a versão anterior ao delete marker, restaurando o arquivo localmente
3	<code>aws s3 sync files s3://<bucket>/files/</code>	Re-sincronizar o diretório local com o bucket, repropagando o arquivo restaurado

```
# Passo 1 - Listar versões de file1.txt para identificar o VersionId original
aws s3api list-object-versions \
  --bucket <s3-bucket-name> \
  --prefix files/file1.txt
# Saída: bloco 'DeleteMarkers' (exclusão) + bloco 'Versions' (versão original com VersionId)

# Passo 2 - Baixar a versão anterior usando o VersionId identificado
aws s3api get-object \
  --bucket <s3-bucket-name> \
  --key files/file1.txt \
  --version-id <VERSION-ID> \
```

```
files/file1.txt

# Confirmar restauração local
ls files/

# Passo 3 – Re-sincronizar com o bucket: file1.txt volta ao bucket
aws s3 sync files s3://<s3-bucket-name>/files/
aws s3 ls s3://<s3-bucket-name>/files/
# Saída: file1.txt, file2.txt, file3.txt – restauração completa confirmada
```

Recuperação confirmada: O arquivo file1.txt foi recuperado com sucesso a partir do historial de versões do S3. O versionamento preservou o conteúdo original mesmo após a exclusão propagada pelo `--delete`. O bucket voltou ao estado completo com os 3 arquivos após a re-sincronização.

4. Ferramentas

Amazon EBS	Amazon S3	Amazon EC2
AWS CLI	Python 3	cron / crontab
S3 Versioning	EC2 Instance Connect	AWS IAM (Instance Profile)
s3api list-object-versions	s3api get-object	EC2 Waiters (CLI)

5. Resultado

As duas frentes de gerenciamento de armazenamento foram implementadas e validadas com sucesso. Na frente de snapshots EBS: o snapshot manual foi criado com consistência garantida pela parada da instância; o cron gerou múltiplos snapshots automaticamente; e o `snapshotter_v2.py` aplicou a política de retenção corretamente, reduzindo os snapshots acumulados a exatamente 2 — os mais recentes.

Na frente S3: os 3 arquivos foram sincronizados com o bucket versionado, a exclusão acidental de file1.txt foi propagada com `--delete` e o arquivo foi recuperado via `list-object-versions` + `get-object` com `VersionId` específico. A re-sincronização restaurou o arquivo também no bucket, confirmando o ciclo completo de proteção e recuperação.

O resultado demonstra na prática duas competências complementares que todo profissional de infraestrutura em nuvem precisa dominar: automatizar a proteção de dados para que ela aconteça sem intervenção humana, e saber reverter situações de perda de dados quando a automação ou o operador comete um erro.

6. Aprendizados

Este projeto consolidou aprendizados sobre automação, política de retenção e recuperação de dados que têm aplicação direta em ambientes produtivos. Os quatro mais relevantes dizem respeito a decisões que, se tomadas erradas, geram custos inesperados ou perda permanente de dados.

6.1 Waiters tornam scripts de infraestrutura confiáveis

O encadeamento de operações assíncronas na AWS é um dos pontos onde scripts de infraestrutura amadores falham: o comando de criação de snapshot retorna imediatamente, mas o snapshot pode levar minutos para completar. Um script que reinicia a instância sem aguardar a conclusão do snapshot arrisca perder o backup justamente no momento em que ele é mais necessário.

O padrão correto — e o que foi aplicado neste projeto — é usar `aws ec2 wait snapshot-completed` antes de qualquer operação que dependa do snapshot estar concluído. O waiter implementa polling interno com backoff exponencial, libera o terminal apenas quando o estado desejado é confirmado e elimina a necessidade de loops manuais com `sleep`. É o mecanismo que separa um script funcional de um script confiável.

6.2 --delete no s3 sync é uma faca de dois gumes

A flag `--delete` torna o `s3 sync` uma ferramenta de espelhamento: o bucket fica idêntico ao diretório local após cada execução, incluindo as exclusões. Para deploys de aplicações web estáticas, isso é exatamente o comportamento desejado — versões antigas de arquivos são removidas automaticamente do bucket.

Para backups, é o comportamento errado: se o objetivo é preservar histórico, usar `--delete` significa que uma exclusão acidental local destrói o backup no S3 também. A distinção entre os dois casos de uso precisa ser clara antes de escolher o comando — e o versionamento do S3 é a camada de segurança que torna o erro recuperável quando `--delete` é usado inadvertidamente.

6.3 O S3 não tem 'restaurar versão' — mas tem o workaround correto

A ausência de um comando direto de restauração no S3 surpreende quem vem de sistemas de backup tradicionais. A razão é arquitetural: o S3 é um armazenamento de objetos imutáveis com versionamento — a 'restauração' é, na prática, baixar a versão desejada e re-enviá-la como a versão mais atual.

O fluxo de três passos — `list-object-versions` para identificar o `VersionId`, `get-object` para baixar a versão correta, `sync` para repropagar — é o procedimento padrão. Existe uma alternativa mais direta: remover o delete marker com `delete-object --version-id <id-do-marker>`, o que faz a versão anterior se tornar visível novamente sem necessidade de re-upload. Ambas as abordagens são válidas; a escolha depende de se o arquivo precisa ser modificado antes de ser restaurado.

6.4 Versionamento S3 tem custo — e precisa de política de expiração

Habilitar o versionamento no S3 é uma das decisões mais simples e impactantes de segurança de dados — mas não é gratuita. Cada versão de cada objeto ocupa espaço e gera cobrança como um

objeto independente. Em buckets com alta rotatividade de arquivos ou muitas versões acumuladas, o custo pode crescer significativamente acima do esperado.

A prática correta é sempre complementar o versionamento com S3 Lifecycle Rules que expirem versões antigas automaticamente — por exemplo, manter versões por 30 dias e excluir delete markers após a expiração das versões associadas. Habilitar versionamento sem lifecycle é o equivalente de tirar snapshots sem política de retenção: o custo cresce indefinidamente até alguém perceber na fatura.

Conexão com o `snapshotter_v2.py`: O mesmo princípio da política de retenção implementada em Python para snapshots EBS se aplica ao versionamento S3 — apenas com a configuração nativa de Lifecycle Rules substituindo o script. Em ambos os casos, o objetivo é o mesmo: manter o histórico necessário para recuperação sem acumular custos de versões que não têm mais utilidade operacional.